

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC SƯ PHẠM
KHOA: TOÁN – TIN



BÀI BÁO CÁO THỰC HÀNH SỐ 2
MÔN : LẬP TRÌNH MẠNG
CHỦ ĐỀ:

LẬP TRÌNH SOCKET VỚI PYTHON (CHƯƠNG 2)



Sinh viên thực hiện

: Trương Quốc Anh

Lê Văn Bình

Bùi Xuân Khánh

Đặng Nhật Thanh

Trần Quốc Trường

Lớp

24CNTT3

Nhóm

05

PHẦN A: CẬP ECHO SERVER – CLIENT TCP VÀ QUAN SÁT KẾT NỐI

Cổng dịch vụ quy định: 9005 (Theo công thức: 9000+ZZ, với ZZ=05)

1. Cài đặt đầy đủ

Hệ thống được lập trình bằng ngôn ngữ Python với thư viện chuẩn socket. Cả hai chương trình đều tuân thủ các quy chuẩn kỹ thuật bắt buộc: thiết lập cờ SO_REUSEADDR, quản lý tài nguyên tự động qua cấu trúc with, cơ chế settimeout(), kiểm tra ngắt kết nối an toàn (b""), và xử lý biệt lập từng lớp ngoại lệ mạng.

a. Mã nguồn Server (echo_server.py)

```
import socket

# Cấu hình cổng cho Nhóm 05: 9000 + 05 = 9005
HOST = ""          # Bind trên mọi interface mạng khả dụng (0.0.0.0)
PORT = 9005

def run_server():
    # Sử dụng with cho socket lắng nghe
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        # 1. setsockopt đặt TRƯỚC bind() để tái sử dụng cổng ngay sau khi tắt server
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

        s.bind((HOST, PORT))
        s.listen(5)
        print(f"[*] Server đang lắng nghe trên cổng {PORT}...")
        print(f"[*] Socket lắng nghe: getsockname = {s.getsockname()}")

    while True:
        conn, addr = s.accept()
        # 2. Sử dụng with cho socket phiên làm việc của từng client
        with conn:
            print(f"\n[+] Kết nối mới từ đối tác: {addr}")
            print(f"    - conn.getsockname() = {conn.getsockname()}")
            print(f"    - conn.getpeername() = {conn.getpeername()}")

            while True:
                try:
                    data = conn.recv(1024)
                    # 3. Kiểm tra đối tác đóng kết nối (recv trả về b'')
                    if not data:
                        print(f"[-] Client {addr} đã đóng kết nối bình thường.")
                        break

                    # 4. Giải mã UTF-8 và in thông điệp
                    text = data.decode("utf-8")
                    print(f"    <- Nhận ({len(data)} bytes): {text}")

                    # Gửi vọng lại dữ liệu cho client (Mã hóa UTF-8)
                    conn.sendall(data)

            except OSError as e:
                print(f"[!] Lỗi kết nối với client {addr}: {e}")
                break
```

```
if __name__ == "__main__":  
    run_server()
```

b. Mã nguồn Client (echo_client.py)

Client tạo socket TCP kết nối tới server tại 127.0.0.1:9005, thiết lập cờ `settimeout(5.0)` chống treo mạng. Chương trình nhận chuỗi ký tự từ bàn phím qua `input()`, mã hóa UTF-8 và gửi sang server bằng `sendall()`, sau đó nhận chuỗi vọng về qua `recv(1024)` và giải mã hiển thị lên màn hình. Nhập 'exit' để đóng kết nối an toàn.

```
import socket
```

```
SERVER_IP = "127.0.0.1" # Hoặc IP LAN khi kiểm thử liên máy  
PORT = 9005
```

```
def run_client():  
    # Sử dụng with cho socket kết nối phía client  
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:  
        # 1. Đặt settimeout để tránh treo vô hạn khi mạng gặp sự cố  
        s.settimeout(5.0)  
  
        try:  
            print(f"[*] Đang kết nối tới server {SERVER_IP}:{PORT}...")  
            s.connect((SERVER_IP, PORT))  
            print("[+] Đã kết nối thành công tới server.")  
  
            while True:  
                msg = input("\nNhập chuỗi gửi tới server (gõ 'exit' để thoát): ")  
                if msg.lower() == "exit":  
                    break  
  
                # 2. Mã hóa chuỗi sang byte theo chuẩn UTF-8  
                data_send = msg.encode("utf-8")  
                s.sendall(data_send)  
  
                # 3. Nhận phản hồi và xử lý kiểm tra b""  
                data_recv = s.recv(1024)  
                if not data_recv:  
                    print("[!] Server đã chủ động ngắt kết nối.")  
                    break  
  
                print(f"[Server phản hồi]: {data_recv.decode('utf-8')}")  
  
                # 4. Bắt riêng lỗi không kết nối được tới server (Server chưa bật hoặc sai cổng)  
            except ConnectionRefusedError:  
                print("[!] Lỗi: Không thể kết nối tới Server (ConnectionRefusedError). Hãy kiểm tra xem Server đã bật chưa.")  
  
                # 5. Bắt riêng lỗi hết thời gian chờ  
            except socket.timeout:  
                print("[!] Lỗi: Quá thời gian chờ phản hồi từ phía server (Timeout).")  
  
                # 6. Bắt các lỗi socket tầng hệ điều hành khác  
            except OSError as e:  
                print(f"[!] Ngoại lệ hệ điều hành phát sinh (OSError): {e}")  
  
if __name__ == "__main__":  
    run_client()
```

2. Chạy trên hai máy

Môi trường thực nghiệm:

- Máy A (Server): Chạy chương trình echo_server.py, cấu hình bind trên 0.0.0.0 và lắng nghe tại cổng 9005. Địa chỉ IP nội bộ qua Wi-Fi/Hotspot: 172.20.10.2.
- Máy B (Client): Cùng mạng LAN, chạy tiến trình echo_client.py kết nối tới đích 172.20.10.2:9005. Địa chỉ IP nội bộ: 172.20.10.5.

Kết quả quan sát và đánh giá:

Tiến trình Client trên Máy B bắt tay TCP thành công với Server trên Máy A. Hệ điều hành cấp phát cổng tạm thời 56737 cho Client. Giao tiếp hai chiều diễn ra ổn định: Máy A nhận trọn vẹn chuỗi Xin chào Server tu may 172.20.10.5 (34 bytes) và phản hồi vọng về chính xác sang Máy B.

```
PS D:\ltn> python echo_server.py
[*] Server đang lắng nghe tại cổng 9005...
[*] Socket lắng nghe (s.getsockname()): ('0.0.0.0', 9005)

[+] Đã kết nối với client từ: ('172.20.10.5', 56737)
- Socket phiên cục bộ (conn.getsockname()): ('172.20.10.2', 9005)
- Địa chỉ đối phương (conn.getpeername()): ('172.20.10.5', 56737)
[Nhận từ 56737]: Xin chào Server tu may 172.20.10.5

PS D:\LapTrinh_Mang> & "C:\Users\QUOC ANH\AppData\Local\Programs\Python\Python313\python.exe" d:/LapTrinh_Mang/echo_client.py
[!] Lỗi kết nối: timed out
PS D:\LapTrinh_Mang> python echo_client.py
[+] Kết nối thành công tới 172.20.10.2:9005!

=====
GỢI Ý THỰC HIỆN THÍ NGHIỆM (c):
- Nhập câu tiếng Việt: Đại học Sư phạm Đà Nẵng
- Nhập lệnh '5000' : Tự động gửi chuỗi 5000 ký tự 'A'
- Nhấn phím Enter : Thoát chương trình
=====
Nhập dữ liệu gửi: Xin chào Server tu may 172.20.10.5

--- KẾT QUẢ ĐO ĐẠC ---
[*] Số ký tự chuỗi (len(text)) : 34 ký tự
[*] Số bytes mã hóa (len(text.encode())): 34 bytes
-> Lần recv(1024) thứ 1: nhận được 34 bytes
[*] Tổng số lần client gọi recv(1024): 1 lần
[*] Tổng số bytes nhận về đầy đủ : 34 bytes
[*] Dữ liệu phản hồi : Xin chào Server tu may 172.20.10.5

=====
GỢI Ý THỰC HIỆN THÍ NGHIỆM (c):
- Nhập câu tiếng Việt: Đại học Sư phạm Đà Nẵng
- Nhập lệnh '5000' : Tự động gửi chuỗi 5000 ký tự 'A'
- Nhấn phím Enter : Thoát chương trình
=====
Nhập dữ liệu gửi: 
```

3. Phân tích Bộ năm định danh (Five-tuple) và Giám sát qua netstat

Khi Server hoạt động và tiếp nhận đồng thời 3 tiến trình Client kết nối đến, kết quả kiểm tra bảng kết nối mạng bằng lệnh netstat -an | findstr 9005 ghi nhận trạng thái:

```
PS D:\itm> netstat -an | findstr 9005
TCP 0.0.0.0:9005 0.0.0.0:0 LISTENING
TCP 0.0.0.0:9005 0.0.0.0:0 LISTENING
TCP 127.0.0.1:9005 127.0.0.1:54704 ESTABLISHED
TCP 127.0.0.1:9005 127.0.0.1:63392 ESTABLISHED
TCP 127.0.0.1:9005 127.0.0.1:64470 ESTABLISHED
TCP 127.0.0.1:54704 127.0.0.1:9005 ESTABLISHED
TCP 127.0.0.1:63392 127.0.0.1:9005 ESTABLISHED
TCP 127.0.0.1:64470 127.0.0.1:9005 ESTABLISHED
```

Bảng 1. Phân tích bộ năm định danh (Five-tuple) của các kết nối qua netstat

Giao thức	Địa chỉ IP cục bộ	Cổng cục bộ	Địa chỉ IP đối tác	Cổng đối tác	Trạng thái	Vai trò của socket
TCP	0.0.0.0	9005	0.0.0.0	0	LISTENING	Socket lắng nghe (Server) chờ kết nối
TCP	127.0.0.1	9005	127.0.0.1	54704	ESTABLISHED	Socket phiên Server phục vụ Client 1
TCP	127.0.0.1	9005	127.0.0.1	63392	ESTABLISHED	Socket phiên Server phục vụ Client 2
TCP	127.0.0.1	9005	127.0.0.1	64470	ESTABLISHED	Socket phiên Server phục vụ Client 3
TCP	127.0.0.1	54704	127.0.0.1	9005	ESTABLISHED	Socket kết nối phía Client 1
TCP	127.0.0.1	63392	127.0.0.1	9005	ESTABLISHED	Socket kết nối phía Client 2
TCP	127.0.0.1	64470	127.0.0.1	9005	ESTABLISHED	Socket kết nối phía Client 3

- Cơ chế Server phân biệt các Client không bị nhầm lẫn:

Mỗi kết nối TCP được định danh duy nhất trên toàn hệ thống bằng **Bộ năm (Five-tuple)**:

{Giao thức, IP nguồn, Cổng nguồn, IP đích, Cổng đích}

Dù các Client đều gửi tới cùng đích 127.0.0.1:9005, nhân hệ điều hành đã tự động cấp phát các cổng tạm thời (*ephemeral ports*) khác nhau gồm **54704**, **63392** và **64470**.

Nhờ cổng nguồn độc lập, bộ năm của mỗi kết nối là duy nhất. Hệ điều hành dựa vào bộ năm này để chuyển đúng gói tin vào bộ đệm nhận của từng socket phiên tương ứng mà không xảy ra xung đột hay nhầm lẫn dữ liệu.

4. Phân biệt Socket lắng nghe (Listening Socket) và Socket phiên (Session Socket)

- **Socket lắng nghe (s):** `getsockname() = ('0.0.0.0', 9005)`. Không có `getpeername()` do socket chưa ghép cặp với đầu cuối nào. Socket này được tạo bởi `bind()` và `listen()`, đóng vai trò thường trực đón nhận các yêu cầu kết nối từ hàng đợi (*backlog queue*); không tham gia truyền nhận dữ liệu ứng dụng (`recv()` sẽ báo lỗi).
- **Socket phiên (conn):** `getsockname() = ('127.0.0.1', 9005)` và `conn.getpeername() = ('127.0.0.1', 54704)`. Được sinh động sau khi `accept()` hoàn tất bắt tay ba bước. Sở hữu bộ đệm gửi/nhận riêng biệt để đọc/ghi dữ liệu hai chiều (`recv()` / `sendall()`). Việc đóng conn chỉ kết thúc phiên làm việc với client đó mà không ảnh hưởng tới tiến trình lắng nghe chung của server.

5. Kết quả và phân tích ba thí nghiệm thực nghiệm

a. Thí nghiệm (a): Chạy Client khi Server chưa khởi động

- **Hiện tượng:** Client dừng ngay lập tức và ném ngoại lệ `ConnectionRefusedError: [WinError 10061] No connection could be made because the target machine actively refused it`.
- **Nguyên nhân:** Gói tin [SYN] gửi tới cổng 9005 nhưng nhân hệ điều hành máy đích không thấy tiến trình nào đang ở trạng thái LISTENING, do đó tự động phản hồi lại phân đoạn mang cờ [RST, ACK] từ chối kết nối. Tầng socket của Python nhận cờ RST và phát sinh ngoại lệ.

b. Thí nghiệm (b): Hiện tượng TIME_WAIT và vai trò SO_REUSEADDR

```
PS D:\ltm> python echo_server.py
Traceback (most recent call last):
  File "D:\ltm\echo_server.py", line 56, in <module>
    run_server()
  File "D:\ltm\echo_server.py", line 13, in run_server
    s.bind((HOST, PORT))
OSError: [WinError 10048] Only one usage of each socket address (protocol/network address/port) is normally permitted
```

- **Hiện tượng:** Vô hiệu hóa dòng cờ `SO_REUSEADDR`, chạy server, cho client kết nối rồi ngắt server đột ngột (Ctrl + C). Khởi động lại ngay lập tức thì server báo lỗi: `OSError: [WinError 10048] Only one usage of each socket address is normally permitted`.

(Chèn ảnh chụp lỗi: *image_d8576b.png*)

- **Nguyên nhân:** Khi Server chủ động ngắt kết nối trước (*Active Close*), cổng rơi vào trạng thái TIME_WAIT (kéo dài từ 30 giây đến 2 phút) theo quy chuẩn TCP để chờ tiêu biến hết các gói tin cũ đi lạc. Bật SO_REUSEADDR chỉ thị cho hệ điều hành cho phép tái sử dụng cổng ngay lập tức.

c. Thí nghiệm (c): Kiểm thử mã hóa tiếng Việt và luồng dữ liệu lớn

- **Đo đạc chuỗi tiếng Việt ("Đại học Sư phạm Đà Nẵng"):**

- Kết quả: Số ký tự văn bản $\text{len}(\text{text}) = 23$ ký tự; dung lượng bytes sau mã hóa $\text{len}(\text{text.encode("utf-8")}) = 35$ bytes (tăng thêm 12 bytes). Client nhận đủ dữ liệu trong 1 lần gọi $\text{recv}(1024)$.
- *Nguyên nhân:* Các ký tự tiếng Việt có dấu thuộc bảng mã Unicode mở rộng, chuẩn UTF-8 sử dụng 2 đến 3 bytes để biểu diễn (thay vì 1 byte như mã ASCII). Vì vậy trong lập trình mạng, kích thước bộ đệm luôn phải tính theo số byte thực tế.

```
Nhập dữ liệu gửi: Đại học Sư phạm Đà Nẵng

--- KẾT QUẢ ĐO ĐẠC ---
[*] Số ký tự chuỗi (len(text)) : 23 ký tự
[*] Số bytes mã hóa (len(text.encode)): 35 bytes
    -> Lần recv(1024) thứ 1: nhận được 35 bytes
[*] Tổng số lần client gọi recv(1024): 1 lần
[*] Tổng số bytes nhận về đầy đủ : 35 bytes
[*] Dữ liệu phản hồi : Đại học Sư phạm Đà Nẵng
```

- **Đo đạc chuỗi lớn (5000 ký tự 'A'):**

- Kết quả: Khối dữ liệu 5000 bytes khiến Client phải lặp hàm $\text{recv}(1024)$ đúng 5 lần mới nhận đủ ($4 \text{ lần} \times 1024 \text{ bytes} + 1 \text{ lần} \times 904 \text{ bytes}$). Phía Server cũng ghi nhận 5 lần nhận dữ liệu tương ứng.
- *Nguyên nhân:* TCP hoạt động theo cơ chế luồng byte liên tục (*byte-stream*), không bảo toàn ranh giới thông điệp. Tham số 1024 chỉ định dung lượng tối đa đọc từ bộ đệm của hệ điều hành trong một lần gọi.

```
Nhập dữ liệu gửi: 5000

--- KẾT QUẢ ĐO ĐẠC ---
[*] Số ký tự chuỗi (len(text)) : 5000 ký tự
[*] Số bytes mã hóa (len(text.encode)): 5000 bytes
    -> Lần recv(1024) thứ 1: nhận được 1024 bytes
    -> Lần recv(1024) thứ 2: nhận được 1024 bytes
    -> Lần recv(1024) thứ 3: nhận được 1024 bytes
    -> Lần recv(1024) thứ 4: nhận được 1024 bytes
    -> Lần recv(1024) thứ 5: nhận được 904 bytes
[*] Tổng số lần client gọi recv(1024): 5 lần
[*] Tổng số bytes nhận về đầy đủ : 5000 bytes
[*] Dữ liệu phản hồi : [Chuỗi 5000 ký tự đã nhận đủ]
```

PHẦN B: GIAO THỨC ĐĂNG NHẬP CHAT (CHAT LOGIN PROTOCOL)

1. Server chat_login_server.py

Chương trình server được xây dựng theo kiến trúc tuần tự (iterative server), lắng nghe tại cổng 9005 (Nhóm 05). Server tiếp nhận kết nối, duy trì tập hợp `active_users` để quản lý các tài khoản đang hoạt động, và giải phóng phiên khi nhận được lệnh `QUIT` hoặc khi đối phương chủ động ngắt kết nối (`recv()` trả về `b''`).

Mã nguồn chương trình Server:

```
import socket

HOST = ""
PORT = 9005 # Cổng Nhóm 05

# Tập hợp lưu trữ danh sách username đang online trong phiên hiện tại
active_users = set()

# Bảng mã lỗi chuẩn
ERR_FORMAT = "1" # Sai định dạng (rỗng, >20 ký tự, chứa '|')
ERR_EXISTS = "2" # Tên đã tồn tại
ERR_SEQUENCE = "3" # Gửi lệnh khác khi chưa LOGIN hợp lệ

def recv_line(sock, buf=b''):
    """
    Hàm đọc từ socket theo ranh giới b'\n',
    trả về (chuỗi_dòng, bộ_đệm_còn_lại) hoặc (None, buf) khi đóng kết nối.
    """
    while b"\n" not in buf:
        chunk = sock.recv(1024)
        if not chunk:
            return None, buf
        buf += chunk
    line, _, rest = buf.partition(b"\n")
    return line.decode("utf-8"), rest

def run_server():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((HOST, PORT))
        s.listen(5)
        print(f"[*] SERVER CHAT LOGIN (TUẦN TỰ) ĐANG CHẠY TẠI CỔNG {PORT}...")

        while True:
            # Server tuần tự tiếp nhận lần lượt từng client
            conn, addr = s.accept()
            print(f"\n[+] Tiếp nhận kết nối từ: {addr}")

            with conn:
                buf = b''
                logged_in_user = None

                while True:
                    line, buf = recv_line(conn, buf)
                    if line is None:
                        print(f"[-] Client {addr} đã đóng kết nối đột ngột.")
                        break
```

```

line = line.strip()
if not line:
    continue

print(f"    <- Nhận từ {addr}: {repr(line)}")

# 1. Kiểm tra lệnh QUIT
if line == "QUIT":
    conn.sendall(b"OK|Tam biet\n")
    print(f"[*] Client {addr} đã gửi QUIT.")
    break

# 2. Xử lý thông điệp LOGIN
if line.startswith("LOGIN|"):
    name = line[6:] # Lấy chuỗi sau tiền tố LOGIN|

    # Kiểm tra các điều kiện ràng buộc
    if len(name) == 0:
        resp = f"ERR|{ERR_FORMAT}|Ten không được để trống\n"
    elif len(name) > 20:
        resp = f"ERR|{ERR_FORMAT}|Ten không được quá 20 ký tự\n"
    elif "|" in name or "\n" in name:
        resp = f"ERR|{ERR_FORMAT}|Ten chưa ký tự cam\n"
    elif name in active_users:
        resp = f"ERR|{ERR_EXISTS}|Ten đã tồn tại\n"
    else:
        logged_in_user = name
        active_users.add(name)
        resp = f"OK|Chào {name}\n"
        print(f"[+] User [{name}] đăng nhập thành công. Active users:
{active_users}")

    conn.sendall(resp.encode("utf-8"))

else:
    # Gửi lệnh khác khi chưa LOGIN hợp lệ
    resp = f"ERR|{ERR_SEQUENCE}|Phải LOGIN trước khi thực hiện lệnh
khac\n"

    conn.sendall(resp.encode("utf-8"))

# Giải phóng tài khoản khi client kết thúc phiên
if logged_in_user and logged_in_user in active_users:
    active_users.remove(logged_in_user)
    print(f"[-] Đã giải phóng user [{logged_in_user}]. Active users:
{active_users}")

print(f"[*] Hoàn tất phục vụ {addr}. Sẵn sàng đón client tiếp theo.")

if __name__ == "__main__":
    run_server()

```

2. Client chat_login_client.py

Client cho phép người dùng nhập tên hiển thị từ bàn phím, đóng gói thông điệp theo cú pháp LOGIN|<tên>\n và phân tích phản hồi từ server. Nếu server trả về ERR, client đếm số lần và cho phép người dùng nhập lại tối đa 3 lần trước khi tự đóng phiên làm việc.

Mã nguồn chương trình Client:

```

import socket

SERVER_IP = "127.0.0.1"
PORT = 9005

def recv_line(sock, buf=b''):
    while b'\n' not in buf:
        chunk = sock.recv(1024)
        if not chunk:
            return None, buf
        buf += chunk
    line, _, rest = buf.partition(b'\n')
    return line.decode("utf-8"), rest

def run_client():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        try:
            print(f"[*] Đang kết nối tới Server {SERVER_IP}:{PORT}...")
            s.connect((SERVER_IP, PORT))
            print("[+] Kết nối thành công!")

            buf = b''
            attempts = 0
            login_success = False

            # Vòng lặp cho phép nhập lại tối đa 3 lần
            while attempts < 3 and not login_success:
                attempts += 1
                name = input(f"\nNhập tên đăng nhập (Lần thử {attempts}/3): ").strip()

                msg = f"LOGIN|{name}\n"
                s.settimeout(5.0)
                s.sendall(msg.encode("utf-8"))

                resp, buf = recv_line(s, buf)
                s.settimeout(None)

                if resp is None:
                    print("[!] Server đã ngắt kết nối.")
                    return

                print(f"[SERVER PHẢN HỒI] -> {resp}")

                if resp.startswith("OK|"):
                    login_success = True
                    print("[v] ĐĂNG NHẬP THÀNH CÔNG!")
                elif resp.startswith("ERR|"):
                    parts = resp.split("|", 2)
                    err_code = parts[1] if len(parts) > 1 else "?"
                    err_desc = parts[2] if len(parts) > 2 else "Lỗi không xác định"
                    print(f"[x] Đăng nhập thất bại (Mã lỗi {err_code}): {err_desc}")

            if not login_success:
                print("\n[!] Bạn đã nhập sai quá 3 lần. Đang đóng kết nối...")
                s.sendall(b"QUIT\n")
            else:
                print("\n(Gõ 'QUIT' để thoát)")

```

```

while True:
    cmd = input("Lệnh tiếp theo > ").strip()
    if not cmd:
        continue
    s.sendall((cmd + "\n").encode("utf-8"))
    resp, buf = recv_line(s, buf)
    print(f"[SERVER PHẢN HỒI] -> {resp}")
    if cmd == "QUIT":
        break

except ConnectionRefusedError:
    print("[!] Lỗi: Server chưa chạy hoặc sai cổng!")
except Exception as e:
    print(f"[!] Lỗi phát sinh: {e}")
finally:
    print("[*] Đã đóng phiên làm việc.")

if __name__ == "__main__":
    run_client()

```

3. Xử lý ranh giới thông điệp

a. Cơ sở lý thuyết và giải pháp kỹ thuật

- Bản chất của TCP: TCP là giao thức hướng luồng byte liên tục (*byte-stream*), không tự bảo lưu ranh giới thông điệp tầng ứng dụng. Khi client gửi nhiều thông điệp liên tiếp hoặc gửi một chuỗi chứa nhiều lệnh cùng lúc, hệ điều hành và mạng có thể gom chúng vào một phân đoạn TCP duy nhất.
- Cơ chế phân tách (Framing): Giao thức sử dụng ký tự phân cách ranh giới dòng `\n` để kết thúc mỗi thông điệp.
- Vai trò của hàm `recv_line()`: Hàm duy trì một biến tích lũy `buf`. Dữ liệu nhận từ socket được cộng dồn vào `buf` cho đến khi bắt gặp `\n`. Lúc này, hàm gọi `buf.partition(b"\n")` để cắt lấy đúng một thông điệp trọn vẹn đưa vào xử lý, phần byte dư thừa còn lại tiếp tục được giữ trong `buf` phục vụ cho các lần đọc kế tiếp mà không làm thất lạc dữ liệu.

b. Kịch bản thực nghiệm

Để chứng minh chương trình vẫn chạy đúng khi xảy ra hiện tượng dính thông điệp, chương trình kiểm thử `test_boundary.py` thực hiện gửi đồng thời hai câu lệnh độc lập trong cùng một lần gọi `sendall()` duy nhất:

```
payload = b"LOGIN|alice\nQUIT\n"
```

c. Mã nguồn kịch bản kiểm thử (`test_boundary.py`):

```

import socket

SERVER_IP = "127.0.0.1"
PORT = 9005

def recv_line(sock, buf=b''):
    while b"\n" not in buf:
        chunk = sock.recv(1024)
        if not chunk:
            return None, buf
        buf += chunk

```

```

line, _, rest = buf.partition(b"\n")
return line.decode("utf-8"), rest

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((SERVER_IP, PORT))
    print("[+] Đã kết nối tới Server.")

    # Gửi gộp 2 thông điệp dính liền nhau trong 1 lần gọi sendall duy nhất
    payload = b"LOGIN|alice\nQUIT\n"
    print(f"[*] Client gửi gói gộp (1 lần sendall): {payload}")
    s.sendall(payload)

    # Đọc thông điệp phản hồi thứ nhất
    buf = b""
    resp1, buf = recv_line(s, buf)
    print(f"<- Nhận phản hồi 1: {resp1}")

    # Đọc thông điệp phản hồi thứ hai từ phần bộ đệm còn lại
    resp2, buf = recv_line(s, buf)
    print(f"<- Nhận phản hồi 2: {resp2}")

```

```

PS D:\ltm> python test_boundary.py
[+] Đã kết nối tới Server.
[*] Client gửi gói gộp (1 lần sendall): b'LOGIN|alice\nQUIT\n'
<- Nhận phản hồi 1: OK|Chào alice
<- Nhận phản hồi 2: OK|Tạm biệt

```

4. Bảng mã lỗi

Trong quá trình xây dựng chương trình Chat Login sử dụng giao thức TCP, cần quy định rõ các trường hợp Server trả về lỗi để Client có thể nhận biết và xử lý phù hợp. Phần này thực hiện cập nhật bảng đặc tả thông điệp của hệ thống với các mã lỗi liên quan đến quá trình đăng nhập. Đồng thời tiến hành kiểm thử các trường hợp lỗi để kiểm tra Server có trả về đúng mã lỗi và nội dung thông báo theo đặc tả hay không. Các mã lỗi được xây dựng gồm:

ERR 1: Sai định dạng tên đăng nhập.

ERR 2: Tên đăng nhập đã tồn tại.

ERR 3: Sai trình tự thực hiện lệnh, chưa đăng nhập nhưng đã gửi lệnh khác.

Hệ thống sử dụng cấu trúc phản hồi lỗi:

ERR|Mã_lỗi|Nội_dung_lỗi

Trong đó, mã lỗi giúp Client xác định loại lỗi xảy ra, còn nội dung lỗi giúp người sử dụng biết nguyên nhân cụ thể.

Mã lỗi (<mã>)	Tên mã lỗi	Điều kiện phát sinh tại Server	Ví dụ phản hồi từ Server
1	ERR_FORMAT	Tên đăng nhập để trống, vượt quá 20 ký tự, hoặc chứa ký tự phân cách cấm (, \n).	ERR 1 Ten không được qua 20 ký tự\n ERR 1 Ten chưa ký tu cam\n
2	ERR_EXISTS	Biệt danh đăng nhập đã có client khác sử dụng và đang online trong tập active_users.	ERR 2 Ten đã tồn tại\n
3	ERR_SEQUENCE	Vi phạm thứ tự: Client gửi các lệnh	ERR 3 Phải LOGIN trước khi thực hiện lệnh khác\n

		ng nghiệp vụ khác khi chưa hoàn tất lệnh LOGIN.	
--	--	---	--

Giải thích

- ERR 1 – Sai định dạng tên:
Server kiểm tra tên đăng nhập trước khi cho phép đăng nhập. Nếu tên để trống, dài quá 20 ký tự hoặc chứa ký tự | thì Server từ chối yêu cầu và trả về mã lỗi 1.
- ERR 2 – Tên đã tồn tại:
Server lưu các tên đã đăng nhập trong tập active_users. Nếu Client gửi yêu cầu đăng nhập bằng một tên đã tồn tại trong tập này, Server trả về mã lỗi 2.
- ERR 3 – Sai trình tự:
Client phải thực hiện lệnh LOGIN trước khi thực hiện các lệnh khác. Nếu gửi một lệnh khác khi chưa đăng nhập, Server trả về mã lỗi 3.

5. Kiểm thử và nhận xét

a. Bảng ca kiểm thử thực tế (Bảng 1)

Bảng tổng hợp các kịch bản kiểm thử chức năng xác thực đăng nhập, kiểm tra tính hợp lệ của dữ liệu đầu vào và quản lý trạng thái phiên:

TT	Mục đích	Dữ liệu client gửi	Kết quả mong đợi	Kết quả thực tế / Đạt
1	Đăng nhập hợp lệ	LOGIN nam\n	OK Chào nam\n	OK Chào nam\n — Đạt
2	Tên đã tồn tại	LOGIN nam\n	ERR 2 Ten da ton tai	ERR 2 Ten da ton tai — Đạt
3	Tên không được quá 20 ký tự	LOGIN namcoloi voibanthanahaha hahahah\n	ERR 1 Ten khong duoc qua 20 ky tu	ERR 1 Ten khong duoc qua 20 ky tu — Đạt
4	Tên chứa ký tự cấm	LOGIN truong @\n	ERR 1 Ten chua ky tu cam	ERR 1 Ten chua ky tu cam — Đạt
5	Tên sai định dạng (để trống)	LOGIN \n	ERR 1 Ten khong duoc de trong	ERR 1 Ten khong duoc de trong — Đạt

Đăng nhập hợp lệ

```
D:\HOC\12.LapTrinhMang\pychat>python chat_login_client.py
[*] Đang kết nối tới Server 127.0.0.1:9005...
[+] Kết nối thành công!

Nhập tên đăng nhập (Lần thứ 1/3): nam
[SERVER PHẢN HỒI] -> OK|Chào nam
[v] ĐĂNG NHẬP THÀNH CÔNG!

(Gõ 'QUIT' để thoát)
Lệnh tiếp theo > █
```

Tên đã tồn tại

```
D:\HOC\12.LapTrinhMang\pychat>python chat_login_client.py
[*] Đang kết nối tới Server 127.0.0.1:9005...
[+] Kết nối thành công!

Nhập tên đăng nhập (Lần thứ 1/3): nam
[SERVER PHẢN HỒI] -> ERR|2|Ten da ton tai
[x] Đăng nhập thất bại (Mã lỗi 2): Ten da ton tai

Nhập tên đăng nhập (Lần thứ 2/3): S█
```

Tên sai định dạng (để trống)

```
D:\HOC\12.LapTrinhMang\pychat>python chat_login_client.py
[*] Đang kết nối tới Server 127.0.0.1:9005...
[+] Kết nối thành công!

Nhập tên đăng nhập (Lần thứ 1/3):
[SERVER PHẢN HỒI] -> ERR|1|Ten khong duoc de trong
[x] Đăng nhập thất bại (Mã lỗi 1): Ten khong duoc de trong

Nhập tên đăng nhập (Lần thứ 2/3): █
```

Tên chứa ký tự cấm

```
Nhập tên đăng nhập (Lần thứ 2/3): nam|hahah
[SERVER PHẢN HỒI] -> ERR|1|Ten chua ky tu cam
[x] Đăng nhập thất bại (Mã lỗi 1): Ten chua ky tu cam

Nhập tên đăng nhập (Lần thứ 3/3):
```

Tên không được quá 20 ký tự

```
Nhập tên đăng nhập (Lần thứ 3/3): namcoloivoibanthahahahahah\n
[SERVER PHẢN HỒI] -> ERR|1|Ten khong duoc qua 20 ky tu
[x] Đăng nhập thất bại (Mã lỗi 1): Ten khong duoc qua 20 ky tu

[!] Bạn đã nhập sai quá 3 lần. Đóng kết nối!
[*] Đã đóng phiên làm việc.

D:\HOC\12.LapTrinhMang\pychat>
```

b. Hiện tượng khi hai Client cùng kết nối (Mô hình Server tuần tự)

```
PS D:\ltm> python chat_login_client.py
[*] Đang kết nối tới Server 127.0.0.1:9005...
[+] Kết nối thành công!

Nhập tên đăng nhập (Lần thứ 1/3): nam
[!] Lỗi phát sinh: timed out
[*] Đã đóng phiên làm việc.
```

Do Server được cài đặt theo mô hình tuần tự (iterative server) đơn luồng, ứng dụng chỉ phục vụ duy nhất một kết nối Client tại một thời điểm. Khi Client 1 đang kết nối giữ phiên, Client 2 gửi yêu cầu sẽ được hệ điều hành lưu tạm trong hàng đợi backlog, nhưng Server chưa gọi accept() cho Client 2. Sau 5.0 giây không nhận được dữ liệu phản hồi, phía Client 2 phát sinh ngoại lệ timed out do cơ chế settimeout(5.0) kích hoạt và tự động đóng phiên.

PHẦN C – CÔNG CỤ TRA CỨU DNS VÀ PHÂN TÍCH ĐỊA CHỈ MẠNG

1. Mục tiêu và cơ sở kỹ thuật

- Xác định IP cục bộ không qua DNS: Chương trình sử dụng kỹ thuật mở socket UDP (SOCK_DGRAM) hướng tới địa chỉ công cộng 8.8.8.8:80 mà không truyền dữ liệu thực tế. Qua hàm getsockname(), hệ điều hành tự động truy vấn bảng định tuyến (Routing Table) và trả về chính xác địa chỉ IP của giao tiếp mạng (LAN interface) đang hoạt động kết nối Internet.
- Phân giải thuận và ranh giới giao thức: Sử dụng socket.gethostbyname() để lấy nhanh bản ghi IPv4 (A record), và socket.getaddrinfo() để lấy đầy đủ các bản ghi phân giải (hỗ trợ phân tách cả AF_INET - IPv4 và AF_INET6 - IPv6).
- Phân giải ngược (Reverse DNS / PTR Record): Gọi hàm socket.gethostbyaddr() để ánh xạ ngược từ địa chỉ IP sang định danh máy chủ (FQDN).

- Phân cấp xử lý ngoại lệ: Chương trình bắt lỗi phân cấp: `socket.gaierror` cho các lỗi tra cứu phân giải tên miền tầng ứng dụng, `socket.herror` cho lỗi bản ghi phân giải ngược, và `socket.error` / `OSError` cho các lỗi mạng, lỗi giao vận tầng thấp hơn.

2. Mã nguồn chương trình (dns_lookup.py):

```
import sys
import socket

def get_local_ip():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        try:
            s.connect(("8.8.8.8", 80))
            ip = s.getsockname()[0]
        except OSError:
            ip = "Không xác định được (máy có thể không có kết nối mạng)"
    return ip

def print_header():
    print("=" * 70)
    print("THÔNG TIN MÁY ĐANG CHẠY CHƯƠNG TRÌNH")
    print("=" * 70)
    hostname = socket.gethostname()
    local_ip = get_local_ip()
    print(f"Tên máy (hostname) : {hostname}")
    print(f"Địa chỉ IP LAN      : {local_ip}")
    print()

def lookup_ipv4(domain):
    print("-" * 70)
    print(f"[1] Địa chỉ IPv4 của '{domain}' (gethostbyname)")
    print("-" * 70)
    try:
        ip = socket.gethostbyname(domain)
        print(f"    IPv4: {ip}")
        return ip
    except socket.gaierror as e:
        print(f"    Lỗi phân giải tên miền (gaierror): {e}")
    except OSError as e:
        print(f"    Lỗi hệ thống (OSError) khác: {e}")
    return None

def lookup_getaddrinfo(domain):
    print("-" * 70)
    print(f"[2] Toàn bộ kết quả getaddrinfo('{domain}', 80, SOCK_STREAM)")
    print("-" * 70)
    results = []
    try:
        infos = socket.getaddrinfo(domain, 80, type=socket.SOCK_STREAM)
        for idx, info in enumerate(infos, start=1):
            family, socktype, proto, canonname, sockaddr = info
            family_name = "AF_INET (IPv4)" if family == socket.AF_INET else (
                "AF_INET6 (IPv6)" if family == socket.AF_INET6 else str(family)
            )
            print(f"    Dòng {idx}: họ địa chỉ = {family_name}, sockaddr = {sockaddr}")
            results.append((family_name, sockaddr))
        if not infos:
            print("    Không có kết quả trả về")
    except socket.gaierror as e:
        print(f"    Lỗi phân giải tên miền (gaierror): {e}")
    except OSError as e:
        print(f"    Lỗi hệ thống (OSError) khác: {e}")
    return results
```

```

        print("    Không có kết quả trả về.")
except socket.gaierror as e:
    print(f"    Lỗi phân giải tên miền (gaierror): {e}")
except OSError as e:
    print(f"    Lỗi hệ thống (OSError) khác: {e}")
return results

def reverse_lookup(ip):
    print("-" * 70)
    print(f"[3] Tra ngược địa chỉ IP '{ip}' về tên (gethostbyaddr)")
    print("-" * 70)
    if ip is None:
        print("    Bỏ qua vì không có địa chỉ IPv4 hợp lệ để tra ngược.")
        return
    try:
        name, aliases, addrs = socket.gethostbyaddr(ip)
        print(f"    Tên chính   : {name}")
        print(f"    Bí danh    : {aliases}")
        print(f"    Địa chỉ IP : {addrs}")
    except socket.herror as e:
        print(f"    Không tra ngược được (herror): {e}")
    except socket.gaierror as e:
        print(f"    Lỗi phân giải tên miền (gaierror): {e}")
    except OSError as e:
        print(f"    Lỗi hệ thống (OSError) khác: {e}")

def main():
    if len(sys.argv) != 2:
        print("Cách dùng: python dns_lookup.py <tên_miền>")
        print("Ví dụ    : python dns_lookup.py ued.udn.vn")
        sys.exit(1)

    domain = sys.argv[1]
    print_header()
    print(f"Đang tra cứu thông tin DNS cho tên miền: {domain}\n")
    ipv4 = lookup_ipv4(domain)
    print()
    lookup_getaddrinfo(domain)
    print()
    reverse_lookup(ipv4)
    print()
    print("=" * 70)
    print("HOÀN TẤT TRA CỨU")
    print("=" * 70)

if __name__ == "__main__":
    main()

```

Bảng 2

Tên miền	IPv4	Số dòng getaddrinfo (IPv4 / IPv6)	Tra ngược	Ghi chú
www.ued.udn.vn	IPv4: 171.251.51.198	Dòng 1: họ địa chỉ = AF_INET (IPv4), sockaddr = ('171.251.51.198', 80)	Không có	Tên miền giáo dục Việt Nam

www.google.com	IPv4: 142.250.197.110	Dòng 1: họ địa chỉ = AF_INET (IPv4), sockaddr = ('142.250.197.110', 80)	nchkgai-ai-in- f14.1e100.net	Tra ngược địa chỉ IP về tên
www.lose.abc	Không có	Không có	Không có	Bất thành công lỗi socket.gaierror
www.python.org	Không có	Không có	Không có	Bất thành công lỗi socket.gaierror.

Kết quả

```
PS C:\Users\Admin\source\Lap_Trinh_Mang> python dns_lookup.py ued.udn.vn
=====
THÔNG TIN MÁY ĐANG CHẠY CHƯƠNG TRÌNH
=====
Tên máy (hostname) : DESKTOP-Thanh
Địa chỉ IP LAN      : 192.168.2.43

Đang tra cứu thông tin DNS cho tên miền: ued.udn.vn

-----
[1] Địa chỉ IPv4 của 'ued.udn.vn' (gethostbyname)
-----
IPv4: 171.251.51.198

-----
[2] Toàn bộ kết quả getaddrinfo('ued.udn.vn', 80, SOCK_STREAM)
-----
Dòng 1: họ địa chỉ = AF_INET (IPv4), sockaddr = ('171.251.51.198', 80)

-----
[3] Tra ngược địa chỉ IP '171.251.51.198' về tên (gethostbyaddr)
-----
Không tra ngược được (herror): [Errno 11004] host not found

=====
HOÀN TẤT TRA CỨU
```

```

PS C:\Users\Admin\source\Lap_Trinh_Mang> python dns_lookup.py google.com
=====
THÔNG TIN MÁY ĐANG CHẠY CHƯƠNG TRÌNH
=====
Tên máy (hostname) : DESKTOP-Thanh
Địa chỉ IP LAN      : 192.168.2.43

Đang tra cứu thông tin DNS cho tên miền: google.com

-----
[1] Địa chỉ IPv4 của 'google.com' (gethostbyname)
-----
IPv4: 142.250.197.110

-----
[2] Toàn bộ kết quả getaddrinfo('google.com', 80, SOCK_STREAM)
-----
Dòng 1: họ địa chỉ = AF_INET (IPv4), sockaddr = ('142.250.197.110', 80)

-----
[3] Tra ngược địa chỉ IP '142.250.197.110' về tên (gethostbyaddr)
-----
Tên chính   : nchkgai-in-f14.1e100.net
Bí danh     : []
Địa chỉ IP  : ['142.250.197.110']

=====
HOÀN TẤT TRA CỨU

```

```
PS C:\Users\Admin\source\Lap_Trinh_Mang> python dns_lookup.py lose.abc
```

```
=====
THÔNG TIN MÁY ĐANG CHẠY CHƯƠNG TRÌNH
=====
```

```
Tên máy (hostname) : DESKTOP-Thanh
```

```
Địa chỉ IP LAN      : 192.168.2.43
```

```
Đang tra cứu thông tin DNS cho tên miền: lose.abc
```

```
-----
[1] Địa chỉ IPv4 của 'lose.abc' (gethostbyname)
-----
```

```
Lỗi phân giải tên miền (gaierror): [Errno 11001] getaddrinfo failed
```

```
-----
[2] Toàn bộ kết quả getaddrinfo('lose.abc', 80, SOCK_STREAM)
-----
```

```
Lỗi phân giải tên miền (gaierror): [Errno 11001] getaddrinfo failed
```

```
-----
[3] Tra ngược địa chỉ IP 'None' về tên (gethostbyaddr)
-----
```

```
Bỏ qua vì không có địa chỉ IPv4 hợp lệ để tra ngược.
```

```
=====
HOÀN TẤT TRA CỨU
=====
```

```
PS C:\Users\Admin\source\Lap_Trinh_Mang> python dns_lookup.py python.org
```

```
=====
THÔNG TIN MÁY ĐANG CHẠY CHƯƠNG TRÌNH
=====
```

```
Tên máy (hostname) : DESKTOP-Thanh
```

```
Địa chỉ IP LAN      : 192.168.110.1
```

```
Đang tra cứu thông tin DNS cho tên miền: python.org
```

```
-----
[1] Địa chỉ IPv4 của 'python.org' (gethostbyname)
-----
```

```
Lỗi phân giải tên miền (gaierror): [Errno 11001] getaddrinfo failed
```

```
-----
[2] Toàn bộ kết quả getaddrinfo('python.org', 80, SOCK_STREAM)
-----
```

```
Lỗi phân giải tên miền (gaierror): [Errno 11001] getaddrinfo failed
```

```
-----
[3] Tra ngược địa chỉ IP 'None' về tên (gethostbyaddr)
-----
```

```
Bỏ qua vì không có địa chỉ IPv4 hợp lệ để tra ngược.
```

```
=====
HOÀN TẤT TRA CỨU
=====
```

PHẦN D: CÂU HỎI THẢO LUẬN

CÂU 1: accept() sinh ra một socket mới để làm gì? Nếu server gọi recv() ngay trên socket lắng nghe thì điều gì xảy ra và vì sao?

Trả lời :

Phương thức accept() sinh ra một socket phiên mới (conn) nhằm tách biệt kênh trao đổi dữ liệu riêng với từng client, giúp socket lắng nghe ban đầu (s) tiếp tục rảnh tay quay lại hàng đợi chờ đón các kết nối tiếp theo. Nếu không chia ra hai loại socket này, server sẽ bị nghẽn và không thể phục vụ nhiều client kết nối. Minh chứng thực tế từ kết quả nhóm em chạy ở Phần A (Bảng 1 trong báo cáo): server mở socket lắng nghe cố định tại cổng 9005 với s.getsockname() = ('0.0.0.0', 9005). Khi ba client lần lượt kết nối tới, accept() đã tạo ra ba socket phiên độc lập có cùng IP đích 127.0.0.1:9005 nhưng mang cổng đối tác (conn.getpeername()) hoàn toàn khác nhau là 54704, 63392 và 64470. Nhờ đó, server phân biệt chính xác dữ liệu của từng bạn mà không sợ nhầm lẫn.

Nếu server cố tình gọi s.recv() trực tiếp trên socket lắng nghe, hệ điều hành Windows sẽ ném ngay ngoại lệ: OSError: [WinError 10057] A request to send or receive data was not allowed because the socket is not connected... (hoặc [Errno 107] Transport endpoint is not connected trên Linux). Nguyên nhân là vì socket lắng nghe mới chỉ được gán địa chỉ cục bộ qua bind() để chờ bắt tay ba bước (LISTEN), nó chưa hoàn tất kết nối với một tiến trình cụ thể nào nên chưa có địa chỉ đích trong bộ năm (Five-tuple), khiến tầng vận chuyển (TCP) không thể xác định kênh dữ liệu để đọc hay ghi.

Câu 2: Bạn gọi send() với 100 byte nhưng hàm trả về 20 — điều đó nghĩa là gì? Vì sao trong học phần luôn dùng sendall() ? Có tình huống nào sendall() vẫn chưa đủ để bảo đảm bên nhận đọc đúng thông điệp?

Trả lời:

Khi gọi send() với 100 byte mà hàm chỉ trả về 20, điều đó có nghĩa là hệ điều hành mới chỉ kịp nạp thành công 20 byte đầu vào bộ đệm gửi (send buffer) của socket; 80 byte còn lại chưa được chuyển đi (thường do bộ đệm tầng mạng tạm thời bị đầy hoặc mạng đang nghẽn). Trong học phần luôn yêu cầu dùng sendall() vì hàm này đã được viết sẵn một vòng lặp liên tục gọi send() cho đến khi toàn bộ khối byte được đẩy hết vào bộ đệm hoặc phát sinh lỗi mạng, giúp người lập trình không phải tự viết vòng lặp kiểm tra số byte gửi dở dang.

Tuy vậy, chỉ dùng sendall() thôi thì vẫn **chưa đủ** để bên nhận đọc đúng trọn vẹn thông điệp. Lý do là giao thức TCP truyền dữ liệu dưới dạng luồng byte liên tục (byte-stream), hoàn toàn không lưu giữ ranh giới thông điệp của tầng ứng dụng. Dữ liệu gửi đi rất dễ bị phân mảnh hoặc dính gói trên đường truyền. Minh chứng từ số liệu đo đạc thực tế của nhóm em ở Phần A (mục 5.c): khi gửi chuỗi lớn 5.000 ký tự 'A' (5.000 bytes), bên gửi chỉ gọi duy nhất 1 lần sendall(), nhưng bên nhận bắt buộc phải lặp hàm recv(1024) đúng 5 lần mới gom đủ (gồm 4 lần nhận 1024 bytes và 1 lần nhận 904 bytes). Tương tự ở kịch bản kiểm thử test_boundary.py của Phần B, khi client gửi gộp 2 thông điệp b"LOGIN|alice\nQUIT\n" trong 1 lần sendall(), nếu server không chủ động dùng hàm recv_line() để tách theo ký tự ranh giới \n thì sẽ đọc dính chum cả hai lệnh làm một, dẫn đến sai lệch giao thức.

Câu 3: gethostbyname() và getaddrinfo() khác nhau ở những điểm nào? Vì sao chương trình dùng thật nên dùng getaddrinfo() hoặc socket.create_connection()?

Trả lời :

Điểm khác biệt căn bản: `gethostbyname()` là hàm đời cũ, chỉ hỗ trợ phân giải và trả về duy nhất một địa chỉ IPv4 dưới dạng chuỗi, hoàn toàn không hỗ trợ IPv6 và không cung cấp thông tin về loại socket hay giao thức mạng. Ngược lại, `getaddrinfo()` là chuẩn hiện đại theo RFC 3493, trả về một danh sách các bộ 5 phần tử (family, type, proto, canonname, sockaddr) chứa đầy đủ cả IPv4, IPv6 cùng cấu trúc địa chỉ mạng có thể truyền thẳng vào các hàm `socket()` và `connect()`. Minh chứng từ kết quả chạy file `dns_lookup.py` của bạn Thanh trên máy DESKTOP-Thanh ở Phần C: khi tra cứu tên miền `google.com`, hàm `gethostbyname()` chỉ trả về duy nhất địa chỉ IPv4 142.250.197.110, trong khi `getaddrinfo()` phân tích chi tiết bản ghi theo họ địa chỉ `AF_INET` kèm số cổng 80.

Các chương trình thực tế bắt buộc phải sử dụng `getaddrinfo()` hoặc hàm tiện ích `socket.create_connection()` vì hạ tầng mạng Internet ngày nay vận hành theo mô hình mạng kép (Dual-Stack IPv4/IPv6). Nếu phần mềm chỉ dùng `gethostbyname()` để ép cứng theo `AF_INET`, ứng dụng sẽ sập ngay khi chạy trong các mạng thuần IPv6 hoặc khi đường truyền IPv4 gặp sự cố. Hàm `socket.create_connection()` là giải pháp tối ưu nhất vì nó đã tự động hóa việc duyệt qua từng địa chỉ IP trong danh sách trả về của `getaddrinfo()`, thử kết nối lần lượt cho đến khi thành công kèm thiết lập thời gian chờ (timeout), giúp ứng dụng hoạt động ổn định và tin cậy trên mọi môi trường mạng.

Phụ lục A – Mẫu bảng phân công công việc và tự đánh giá

TT	Họ và tên	Mã sinh viên	Công việc đảm nhận	Đóng góp (%)
1	Bùi Xuân Khánh	3120224073	Nhóm trưởng; Phụ trách Phần A (Echo Server/Client, thực nghiệm mạng LAN, phân tích netstat và 3 thí nghiệm) & Tổng hợp hoàn thiện báo cáo.	20%
2	Trần Quốc Trường	3120224167	Lập trình Server tuần tự cho Phần B, thiết lập bảng mã lỗi (ERR 1, 2, 3), quản lý danh sách <code>active_users</code> và giải phóng phiên	20%
3	Lê Văn Bình	3120224012	Lập trình Client cho Phần B, xây dựng cơ chế thử lại tối đa 3 lần, kiểm thử xử lý dính gói (<code>test_boundary.py</code>) và lập bảng ca kiểm thử	20%
4	Đặng Nhật Thanh	3120224131	Lập trình <code>dns_lookup.py</code> ở Phần C (dò IP qua UDP 8.8.8.8, <code>gethostbyname</code> , <code>getaddrinfo</code> , tra ngược PTR), phân cấp ngoại lệ và thảo luận kết quả.	20%
5	Trương Quốc Anh	3120224005	Phụ trách Phần D; tham gia chạy tiến trình Client trên Máy B phục vụ thực nghiệm liên máy Phần A	20%

Phụ lục B – Nhật ký lỗi (bắt buộc nộp)

TT	Thông báo lỗi (chép nguyên văn)	Tình huống xuất hiện	Nguyên nhân	Cách khắc phục
1	ConnectionRefusedError: [WinError 10061] No connection could be made because the target machine actively refused it	Khi chạy echo_client.py để kết nối tới server tại cổng 9005 trong khi chưa bật echo_server.py (Thí nghiệm a, Phần A).	Phía máy đích không có tiến trình nào đang mở cổng 9005 ở trạng thái LISTENING. Gói tin TCP SYN gửi tới bị ngăn chặn và hệ điều hành phản hồi lại cờ [RST, ACK] từ chối kết nối.	Khởi động file echo_server.py trước, dùng lệnh netstat -an findstr 9005 kiểm tra thấy trạng thái LISTENING rồi mới cho client kết nối.
2	OSError: [WinError 10048] Only one usage of each socket address is normally permitted	Tắt server bằng Ctrl + C ở Thí nghiệm b (khi đã chú thích bỏ dòng SO_REUSEADDR) rồi lập tức gõ lệnh bật lại server ngay	Server là bên chủ động đóng kết nối (Active Close) khiến cổng 9005 bị hệ điều hành giữ lại ở trạng thái TIME_WAIT từ 30s đến 2 phút để xử lý gói tin cũ đi lạc, ngăn cản lệnh bind().	Giữ nguyên câu lệnh cấu hình s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1) đặt ngay trước hàm s.bind() để cho phép tái chiếm dụng cổng ngay lập tức.
3	socket.gaierror: [Errno 11001] getaddrinfo failed	Khi chạy file dns_lookup.py với tên miền không có thực: python dns_lookup.py lose.abc ở Phần C.	DNS Resolver của hệ điều hành gửi truy vấn lên máy chủ phân giải tên miền nhưng nhận mã phản hồi lỗi NXDOMAIN (Non-Existent Domain) do tên miền không tồn tại trên hệ thống DNS.	Bọc các lời gọi gethostbyname() và getaddrinfo() trong khối ngoại lệ try ... except socket.gaierror as e: để in thông báo lỗi thân thiện và chương trình vẫn chạy tiếp.